Report ASG1 NLP

Youssef Awad – 8179
Ahmed Samir – 8120
Mohamed Bassam -8576
Bassem Hesham-8000

Overview

This notebook is **Part 1** of a two-part NLP project. It builds N-gram language models — unigram (1-gram), bigram (2-gram), and trigram (3-gram) — entirely from scratch using Python's standard library (collections.Counter, math, random) and HuggingFace datasets for data loading. The models are trained on a locally stored version of the WikiText corpus (Parquet format), evaluated using perplexity on a validation set for model selection, tested on a held-out test set, and finally used for open-ended text generation. All three trained models are serialized to ngram_models.pkl for re-use by the generation interface.

The notebook is notably **compact** — 9 cells totalling around 100 lines of code — yet covers a complete N-gram pipeline: loading → preprocessing → training → evaluation → generation.

Notebook Structure at a Glance

Cell	Execution Count	Purpose
1	43	Dataset loading
2	44	Text preprocessing
3	45	N-gram model building
4	46	Probability estimation with Laplace smoothing
5	—	Markdown: perplexity formula
6	47	Perplexity computation

Cell	Execution Count	Purpose
7	48	Training, validation, model selection & serialization
8	49	Test set evaluation of best model
9	50	Text generation function definition
10	null	Interactive text generation demo

Section-by-Section Breakdown

Cell 1 — Dataset Loading

The notebook loads a locally stored version of the WikiText corpus from the NgramDataset/ directory using HuggingFace datasets. Three Parquet files are loaded for train, validation, and test splits. The pickle module is also imported at this stage for later model serialization.

The raw data has a column called text containing individual lines from Wikipedia articles. Inspecting the first 5 training rows reveals the format: blank lines, section headings wrapped in = signs, and body paragraphs with extensive punctuation, numbers, and special Wikipedia-style markup (e.g., @-@ for hyphens).

Cell 2 — Text Preprocessing (preprocess_text)

This function accepts a list of raw text lines and an integer n (the n-gram order) and returns a list of token lists ready for n-gram model training.

Steps:

1. **Strip** leading/trailing whitespace from each line
2. **Filter** empty lines and section headings (lines starting with "=")
3. **Lowercase** all remaining text
4. **Remove all punctuation** using str.translate with string.punctuation

5. **Whitespace tokenize** via str.split()
6. **Add boundary tokens:** prepend (n-1) copies of <start> and append one <end> token per sentence

The boundary padding is dynamically scaled to n: unigrams receive 0 <start> tokens, bigrams receive 1, and trigrams receive 2. This makes the same function reusable across all model orders.

What is not done: No stop-word removal, no stemming or lemmatization, no minimum token length filtering, and no handling of WikiText-specific markup artifacts. Wikipedia's @-@ hyphens, for example, are silently collapsed when @ is stripped.

Cell 3 — N-gram Model Building (build_ngram)

This function takes the preprocessed token lists and an order n, and produces three data structures that together constitute the n-gram model.

Data structures returned:

Variable	Type	Description
ngram_counts	Counter	Maps each n-gram tuple to its frequency in training data
context_counts	Counter	Maps each (n-1)-gram context tuple to its total occurrence count
vocab	set	All unique tokens seen in training, including <start> and <end>

For a **unigram model**, the context is always the empty tuple (). This means context_counts[()] accumulates the total number of tokens in the corpus. For bigrams and trigrams, contexts are 1-tuples and 2-tuples respectively.

Cell 4 — Probability Estimation with Laplace Smoothing (get_probability)

The function computes the smoothed conditional probability of a given n-gram:

$$P(w_i \mid w_{i-n+1}^{i-1}) = \frac{\text{count}(w_{i-n+1}^{i}) + 1}{\text{count}(w_{i-n+1}^{i-1}) + |V|}$$

This is standard **Laplace (add-1) smoothing**. The numerator adds 1 to the n-gram count (defaulting to 0 for unseen n-grams), and the denominator adds vocab_size to the context count (defaulting to 0 for unseen contexts). This ensures every probability is strictly positive, preventing zero-probability n-grams during perplexity computation and generation.

- **Unseen n-grams** (OOV): evaluated as 1 / (context_count + vocab_size)
- **Unseen contexts** (fully OOV): denominator becomes vocab_size, yielding a near-uniform distribution

---

Cell 5 — Markdown: Perplexity Formula

The only markdown cell in the notebook defines perplexity and motivates the use of log-space computation:

$$PP = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log\ P(w_i \mid w_{i-1})\right)$$

The formula is presented in its bigram form (P(w_i|w_{i-1})), though the implementation generalises to any n-gram order. The note explains that summing log-probabilities avoids numerical underflow from multiplying many small probabilities together.

⚠️ "Preplexity" is misspelled throughout the notebook (in the markdown header, function name, variable names, and print statements). The correct term is **perplexity**.

Cell 6 — Perplexity Computation (get_preplexity)

This function computes perplexity over a full tokenized dataset given a trained n-gram model.

Algorithm:

1. Iterate over every sentence in the dataset
2. For each n-gram position in the sentence, compute the smoothed log-probability
3. Accumulate the sum of log-probabilities and the total count of n-gram positions

4. Divide to get the average log-probability
5. Return exp(−avg_log_prob)

Counting note: total_words counts n-gram positions — i.e., every sliding window of size n across every sentence — rather than raw word tokens. This means padding tokens (<start>, <end>) contribute to the count, which diverges slightly from standard NLP convention where perplexity is computed only over actual word tokens. This modestly inflates N, slightly deflating reported perplexity.

Cell 7 — Training, Validation & Model Selection

This cell orchestrates the full training and model selection loop, iterating over n ∈ {1, 2, 3}:

1. **Preprocess** training and validation data for the current order n
2. **Train** the model using build_ngram
3. **Validate** using get_preplexity on the validation set
4. **Track** the best model by lowest validation perplexity
5. **Append** each model's parameters (ngram_counts, context_counts, vocab) to a model_parameters list

After the loop, **all three models are serialized** to ngram_models.pkl using pickle.dump. The best model's parameters are separately stored in best_model_parameters (containing ngram_counts, context_counts, and vocab_size as an integer).

Validation Perplexity Results:

Model	Validation Perplexity	Selected?
1-gram (unigram)	**2,114.479**	✅ Best
2-gram (bigram)	9,093.174	❌
3-gram (trigram)	39,922.151	❌

Winner: The unigram model, selected for having the lowest validation perplexity.

⚠️ **Critical observation — counter-intuitive result:** In standard language modelling, higher-order n-gram models are expected to achieve *lower* perplexity by exploiting more context. Here the reverse holds dramatically (the trigram perplexity is ~19× worse than the unigram). This is a well-known failure mode of Laplace smoothing on large, sparse vocabularies:

- WikiText-103 has a very large vocabulary. For bigrams and trigrams, most contexts seen at test time were either unseen or extremely rare in training.
- The +vocab_size term in the denominator becomes so dominant relative to the raw context count that the smoothed probabilities flatten toward uniform — but with a much more penalizing denominator than the unigram has.
- The unigram avoids this: its context is always (), so context_counts[()] is the total corpus token count (a very large number), making the smoothing term +vocab_size relatively small and letting true unigram frequencies dominate.
- More appropriate smoothing techniques (Kneser-Ney, Witten-Bell, or Good-Turing) are specifically designed to handle this sparsity and would reverse the ordering, making higher-order models superior.

Cell 8 — Test Set Evaluation

The best model (unigram) is evaluated on the held-out test set using get_preplexity.

Results:

Split	Perplexity
Validation	2,114.479
Test	**2,231.861**

The ~5.6% increase from validation to test is expected and modest, indicating no meaningful overfitting during the model selection phase.

Cell 9 — Text Generation Function (generate)

This function takes a text prompt and a trained n-gram model and autoregressively generates up to max_length=20 new tokens via **weighted random sampling**.

Algorithm:

1. Preprocess the prompt using the same lowercasing and punctuation-stripping pipeline as training
2. Prepend (n-1) <start> padding tokens
3. At each step:
    - Extract the last (n-1) tokens as the current context
    - Compute the smoothed probability for **every word in the vocabulary** given this context
    - Sample the next word using random.choices with these weights
    - Append the sampled word unless it is <end>, in which case generation stops
4. Return the generated token sequence (stripping the prepended padding)

Cell 10 — Interactive Generation Demo (execution_count: null)

This cell loads all three serialized models from ngram_models.pkl, prompts the user for input via input(), and runs the generate function for each model (n=1, 2, 3), printing the result.

The cell has execution_count: null, meaning it was **never committed in an executed state** — input() cannot run non-interactively. However, a previously captured output is stored in the cell showing results for the prompt "the cat is":

Sample generation outputs (prompt: "the cat is"):

Model	Generated Continuation
1-gram	the cat is dhanno klm rancid porth helipad astonishing desecration construed vagrant mykolaiv harlequin stimi fiftieth ling smythe neolamarckia ᱚᱞᱚᱩ wherewith kipling nicholaievitch
2-gram	the cat is 1703 chandrabindu composted archway wallingford pilasters jinan cyberwarfare rogue 927 flamingos thursday nourish customize improvised ischium 84th pulled shaar interpretive

Model	Generated Continuation
3-gram	the cat is xuri unworthy roca 417 hersey camcorders incident allegorical chihuahua menon psychoactivity 136xe 269 polymerase grates advantages brú unreceptive rfc urocerus

All three outputs are semantically incoherent chains of rare, obscure vocabulary items. This reflects WikiText-103's encyclopedic breadth: rare proper nouns, scientific terms, numbers, foreign words, and place names are all well-represented, and with no real contextual signal (especially in the unigram case), generation degenerates into sampling from the long tail of the vocabulary.

Summary of All Results

Model	Validation Perplexity	Test Perplexity	Selected?
1-gram (unigram)	2,114.479	**2,231.861**	✅ Yes
2-gram (bigram)	9,093.174	—	❌ No
3-gram (trigram)	39,922.151	—	❌ No

Overview

This notebook is **Part 2** of a two-part NLP project (Part 1 being Ngram.ipynb). It implements and benchmarks two classic text classification algorithms — **Multinomial Naive Bayes** and **Logistic Regression (Softmax)** — on the AG News dataset, comparing from-scratch NumPy implementations against scikit-learn equivalents. All evaluation metrics (confusion matrix, precision, recall, F1) are also implemented from scratch and verified against scikit-learn.

Section-by-Section Breakdown

---

§ 2.1 — Dataset: AG News

Source: sh0416/ag_news via 🤗 HuggingFace Datasets

Split	Samples	Features
Train	120,000	label, title, description
Test	7,600	label, title, description

Label Mapping:

Label	Class
1	World
2	Sports
3	Business
4	Sci/Tech

Text preparation: Title and description are concatenated with a space: text = title + " " + description.

---

§ 2.2 — Preprocessing

A custom preprocess_text(text) function performs:

1. **Lowercasing** — all text converted to lowercase
2. **Punctuation removal** — all string.punctuation characters replaced by spaces (e.g., "can't" → "can t")
3. **Whitespace normalization** — multiple spaces collapsed via re.sub(r'\s+', ' ', ...)
4. **Tokenization** — simple whitespace split

Example:

- **Input:** "Wall St. Bears Claw Back Into the Black (Reuters) Reuters - Short-sellers..."

- **Output tokens (24):** ['wall', 'st', 'bears', 'claw', 'back', 'into', 'the', 'black', 'reuters', 'reuters', 'short', 'sellers', ...]

⚠️ **Note:** No stop-word removal is performed. Single-character tokens (e.g., 's', 't', 'n') are retained.

---

§ 2.3 — Naive Bayes Classification

Mathematical Formulation

The notebook lays out the full Multinomial Naive Bayes math:

Class Prior: $P(c) = \frac{N_c}{N}$

Word Likelihood (Laplace smoothing, α=1): $P(w \mid c) = \frac{\text{count}(w,c)+\alpha}{\sum_{w'} \text{count}(w',c)+\alpha \cdot |V|}$

Prediction: $\hat{c} = \arg\max_{c} (\log P(c) + \sum_{w \in d} \text{count}(w,d) \cdot \log P(w \mid c))$

Log-space arithmetic is used throughout to prevent floating-point underflow.

---

§ 2.3.1 — From-Scratch Implementation

Key functions:

- train_multinomial_naive_bayes(train_tokens, train_labels, alpha=1.0) — builds vocabulary, counts, priors, and log-likelihoods; returns a model dict
- predict_naive_bayes(model, test_tokens_list) — scores each document against all classes and returns the argmax

Model dict
keys: vocab, word_to_idx, classes, log_class_priors, log_likelihood, alpha, class_to_idx

Statistics:

- Vocabulary size: **65,015** unique tokens
- Test Accuracy: **90.04%**

⚠️ **Bug noted:** In predict_naive_bayes, the line scores = np.full(n_classes, log_prior) passes the entire log_prior array as a fill value. This works only incidentally

because np.full with an array fill value of the same shape as n_classes broadcasts; it should be scores = log_prior.copy() for clarity and correctness.

---

§ 2.3.2 — Comparison with Scikit-learn

Uses a sklearn.pipeline.Pipeline with:

- CountVectorizer(lowercase=True, token_pattern=r'(?u)\b\w+\b', min_df=1) — matches manual preprocessing
- MultinomialNB(alpha=1.0) — same smoothing

Results:

Model	Accuracy
From-scratch NB	**90.04%**
sklearn MultinomialNB	**90.04%**
Difference	**0.0000**

Sklearn detailed classification report (NB):

Class	Precision	Recall	F1
World	0.9087	0.8958	0.9022
Sports	0.9490	0.9789	0.9637
Business	0.8786	0.8421	0.8600
Sci/Tech	0.8638	0.8847	0.8742
Macro avg	**0.9000**	**0.9004**	**0.9000**

✅ **The from-scratch NB implementation perfectly replicates scikit-learn's results.**

---

§ 2.4 — Logistic Regression

§ 2.4.1 — Feature Representation (Word Bi-grams, Binary)

Rather than unigrams, this section uses **word bi-grams** as features:

- Generated by generate_bigrams(tokens) using "__" as separator (e.g., "wall__st")
- Documents are represented as **binary indicator vectors** (1 if bi-gram present, 0 otherwise), stored as sparse index arrays
- Bi-gram vocabulary built from training set only; test documents use only known bi-grams

Statistics:

- Bi-gram vocabulary size: **1,204,593**
- First document active bi-grams: **23**

§ 2.4.2 — Softmax Logistic Regression (From Scratch, Mini-batch SGD)

Architecture:

- Weight matrix W shape: (num_classes=4, num_features=1,204,593), initialized to zeros
- Bias b shape: (4,), initialized to zeros
- Numerically stable softmax: exp(logits - max(logits))

Training loop (mini-batch SGD):

- Hyperparameters: lr=0.08, epochs=8, batch_size=128, seed=42
- Per-sample gradient: error = p - y_one_hot, accumulated over batch, then averaged
- Gradient update: W -= lr * grad_W, b -= lr * grad_b

Training loss curve:

Epoch	Cross-Entropy Loss
1	1.1351
2	0.8889
3	0.7724

Epoch	Cross-Entropy Loss
4	0.6987
5	0.6458
6	0.6051
7	0.5722
8	0.5448

Test Accuracy: 84.24%

⚠️ **Performance gap noted:** The from-scratch model (84.24%) significantly underperforms vs. sklearn's (90.26–90.32%). Key reasons:

- Only 8 epochs; loss is still decreasing, suggesting the model is under-trained
- No regularization (sklearn uses L2 by default)
- Pure Python inner loop over samples is slow, limiting scalability to more epochs
- learning_rate='optimal' in sklearn's SGD scheduler adapts the rate

§ 2.4.3 — Comparison with Scikit-learn

Models compared:

1. LogisticRegression(solver='saga', max_iter=200, random_state=42) — full solver
2. SGDClassifier(loss='log_loss', alpha=1e-5, max_iter=15, learning_rate='optimal') — SGD variant

Helper: indices_to_csr() converts sparse index lists to scipy.sparse.csr_matrix for sklearn compatibility.

Results:

Model	Accuracy
From-scratch (NumPy SGD)	**84.24%**

Model	Accuracy
sklearn LogisticRegression	**90.26%**
sklearn SGDClassifier	**90.32%**

Sklearn LogisticRegression detailed report:

Class	Precision	Recall	F1
World	0.9258	0.8932	0.9092
Sports	0.9369	0.9689	0.9527
Business	0.8816	0.8579	0.8696
Sci/Tech	0.8664	0.8905	0.8783
Macro avg	**0.9027**	**0.9026**	**0.9024**

---

§ 2.5 — Confusion Matrix & Evaluation Metrics (NumPy From Scratch)

§ 2.5.1 — From-Scratch Implementation

Two functions are implemented using only NumPy:

confusion_matrix_numpy(y_true, y_pred, labels)

- Builds a (n_classes × n_classes) integer matrix
- Rows = true labels, Columns = predicted labels

classification_metrics_from_cm(cm)

- Extracts TP, FP, FN from the confusion matrix diagonals
- Computes per-class precision, recall, F1 with zero-division guard (np.where)
- Returns macro-averaged accuracy, precision, recall, F1

Results for all three models (NumPy computed):

Model	Accuracy	Prec (macro)	Recall (macro)	F1 (macro)
From-scratch LR	0.8424	0.8448	0.8424	0.8422
sklearn LR	0.9026	0.9027	0.9026	0.9024
sklearn SGD	0.9032	0.9033	0.9032	0.9030

Confusion matrices (from-scratch LR):

Code

```
              Pred:World  Sports  Business  Sci/Tech
True: World      1566      101     95       138
True: Sports       44     1756     29        71
True: Business     84       61   1467       288
True: Sci/Tech     79       68    140      1613
```

Most notable confusion: **Business ↔ Sci/Tech** (288 Business predicted as Sci/Tech, 140 Sci/Tech as Business) — these two categories have the most lexical overlap.

---

§ 2.6 — Comparison with Scikit-learn Metrics

The same three model predictions are re-evaluated using sklearn's confusion_matrix, precision_score, recall_score, f1_score, and accuracy_score.

Match verification output (all True):

Code

```
From-scratch Logistic Regression:
  Confusion matrix equal:   True
  Precision per-class match: True
  Recall per-class match:   True
  F1 per-class match:       True
  Accuracy match:           True
```

Precision macro match: True

Recall macro match: True

F1 macro match: True

(Same for sklearn LR and sklearn SGD)

✅ **The from-scratch NumPy evaluation metrics exactly match scikit-learn's implementation across all models.**

Summary of Key Results

Model	Feature Type	Accuracy
Naive Bayes (from scratch)	Unigram counts	**90.04%**
Naive Bayes (sklearn)	Unigram counts	**90.04%**
Logistic Regression (from scratch)	Binary bi-grams	**84.24%**
Logistic Regression (sklearn SAGA)	Binary bi-grams	**90.26%**
SGDClassifier (sklearn, log_loss)	Binary bi-grams	**90.32%**